4-Week Transition Plan: Developer to QA Engineer

Your goal of moving into QA is achievable, especially with your strong development and cloud background. Here's a focused 4-week plan designed for immediate job readiness, featuring two resume-worthy projects and clear daily commitments.

Overall Time Commitment

- **Total daily hours:** 3–4 hours (consistent study and project work)
- **Weeks:** 4 (28 days in total)
- **Total estimate:** ~84–112 hours (enough for a solid transition while working full-time on your job hunt)

Week-by-Week Roadmap

Week 1: Foundations of QA and Manual Testing

Goal: Master core principles and manual QA processes.

- **Daily hours:** 3–4
- **Topics:**
    - Understanding SDLC & STLC
    - QA methodologies (Agile, Waterfall)
    - Types of testing: functional, integration, system, regression, UAT
    - Test case creation, execution, and reporting bugs (use tools like Jira or Trello)
    - Common test documentation: test plans, cases, bug reports
- **Hands-on:**
    - Start *Project 1*: Manual Testing a Simple Web App (E.g., an open-source e-commerce site or your own Flask/Django CRUD app)
        - Write detailed test cases
        - Execute them and document found bugs

Week 2: Test Automation Basics

Goal: Gain automation skills with Selenium and Python (you already know Python!)

- **Daily hours:** 3–4
- **Topics:**
    - Overview of test automation, benefits, and frameworks
    - Selenium basics: locators, interacting with web elements, assertions
    - Writing your first scripts (login, form submission, validations)
    - Basic Page Object Model
- **Hands-on:**
    - Convert some of your manual tests from *Project 1* into Selenium automation scripts

Week 3: Automation Frameworks & API Testing

Goal: Build a small automation suite and practice API testing.

- **Daily hours:** 3–4
- **Topics:**
    - Improving Selenium automation: test structure, data-driven tests
    - Introduction to PyTest/UnitTest for test orchestration
    - Using Postman for API testing (test GET/POST, assertions, chaining)
- **Hands-on:**
    - Build a mini-framework around your scripts from Week 2
    - *Project 2*: API Test Suite (Public APIs like reqres.in or openweather)
        - Test different endpoints, assert responses, document coverage

Week 4: CI/CD Integration, Reporting & Resume Prep

Goal: Demonstrate a "DevOps" mindset: integrate test automation into pipelines and create presentable results.

- **Daily hours:** 3–4
- **Topics:**

- Integrate automation scripts with CI/CD tools (GitHub Actions, Azure Pipelines)
- Automated test execution and reporting (HTML/JUnit reports)
- QA best practices: test coverage, flakiness, environment setup
- Prepare resume, document your projects, portfolio on GitHub
- Practice answering QA interview questions

- **Hands-on:**
  - Set up and demo your automation suite running in CI/CD
  - Polish documentation for both projects:
    - *Project 1*: "Manual & Automated Testing for [App Name]"
    - *Project 2*: "API Test Suite for [API Name]"

Project Ideas for Your Resume

Here are two solid, market-friendly projects:

Project	Skills Highlighted	Outcome / Deliverables
Manual+Automated Testing of Web App	Manual testing, bug reporting, Selenium, test docs, CI/CD	Test cases, bug reports, Python/Selenium code, Test plan, CI pipeline
API Test Suite (with Postman + Py)	API testing, automation, assertions, reporting, PyTest/Unittest	Postman collections, automated scripts, reports, code on GitHub

Tips for a Strong QA Resume and Job Apps

- Show detailed test case and bug report samples.
- Highlight your ability to create both manual and automated tests.
- Link to GitHub repos for both projects.
- Mention familiarity with CI/CD, test planning, and documentation.
- Leverage your developer experience: QA with a "developer's mindset" is a strong selling point.

Final Notes

- Maintain a steady pace (could accelerate if you have more time some days).

- Use weekends to review, polish, or catch up if needed.
- Mock interviews in the last week are highly recommended.
- Stay visible on job boards, update your LinkedIn with "Actively looking—QA roles."

This plan will get you market-ready for junior to mid-level QA positions in a month if you stay disciplined and are hands-on throughout.

Below is a week-wise detailed study plan outlining the topics to be covered and the depth to which each should be studied.

Week - 1

To master Week 1 of your QA and Manual Testing transition (focused on core QA foundations and manual processes), here's a structured breakdown of what you should study and to what depth, tailored for a developer moving into QA:

1. Software Development Life Cycle (SDLC) & Software Testing Life Cycle (STLC)

- **Understand:** Definitions, stages/phases, purpose, and how QA fits in.
- **Study Depth:** Know each phase (requirements, design, development, testing, deployment, maintenance), who's involved, and where testers contribute value.
- **Outcome:** Be able to explain/exemplify both SDLC and STLC in context of a software project.

2. QA Methodologies (Agile, Waterfall)

- **Understand:** What is Agile vs. Waterfall, and their typical processes.
- **Study Depth:** Identify key differences and tester roles. Learn essential Agile concepts (sprints, backlog, ceremonies, stories, cross-functional teams).
- **Outcome:** Able to describe which testing practices fit each methodology and how you would adapt QA work in each.

3. Types of Testing

- **Functional Testing:** Verify individual features as per requirements.
- **Integration Testing:** Ensure components/services work together.

- **System Testing:** Validate the complete, integrated product behaves as expected.
- **Regression Testing:** Retest to confirm new changes didn't break existing functionality.
- **User Acceptance Testing (UAT):** End-user validation of business requirements.
- **Study Depth:** Learn definitions, objectives, and examples for each. Know when and why to apply them.

4. Test Case Creation, Execution, and Bug Reporting

- **Test Cases:** How to write detailed, clear, and reproducible test cases (steps, data, expected results).
- **Execution:** Actually, run your test cases, record results.
- **Bug Reports:** Document issues/defects found – include steps, screenshots, expected/actual results.
- **Study Depth:** Go hands-on—author at least 5-10 sample test cases for a simple app, execute them, and mock a few bug reports.

5. Test Management and Documentation Tools

- **Tools:** Jira, Trello (or alternatives like TestRail, or even Excel/Google Sheets if needed).
- **How:** Create test plans, bug reports, and track test execution—learn the minimal workflow to log and manage test artifacts.
- **Study Depth:** Be able to create example artifacts (test cases, plans, bug reports) in these tools.

6. Key Test Documentation Artifacts

- **Test Plan:** High level—scope, objectives, schedule, deliverables.
- **Test Case:** Detailed steps for what and how to test.
- **Bug Report:** Fields, severity, priority, repro steps.
- **Study Depth:** Create one example of each for your project/hands-on work.

7. Hands-On Project

- **Select a Simple Web App:** Open-source e-commerce OR your own simple Flask/Django CRUD app.
- **What to Do:** Write test cases for main user stories/flows, execute, and document 3–5 bugs found.
- **Outcome:** Portfolio evidence + practical practice. Show that you understand the end-to-end manual QA process.

How Deep Should Your Learning Be?

- Focus on **practical application**: Try to produce at least one real artifact (test case, plan, bug report) for each documentation type.
- Be able to **explain concepts/vs/terminology**: You should be able to talk confidently about various testing types and cycles in an interview or team setting.
- By end of week 1, you should:
    - Know theory and definitions.
    - Be familiar with QA tools.
    - Have initial project/test documentation ready.

Pro Tip

For each major topic, make quick bullet notes and map them to your hands-on project. This helps integrate theory with real practice—the most effective way to transition from dev to QA!

Week -2

To make the most of Week 2 in your 4-week course transitioning from developer to QA—with a focus on test automation using Selenium and Python—here's a structured approach for what you should study and to what depth based on your roadmap:

Week 2: Test Automation Basics with Selenium & Python

1. Understanding Test Automation Fundamentals

- **Topics to Cover:**
    - What is test automation?

    - **Benefits** of automating tests (speed, repeatability, reliability)
    - High-level **automation frameworks** overview (data-driven, keyword-driven, hybrid, BDD)

- **Depth:** Understand the purpose, be able to explain why teams choose automation, and identify which frameworks suit what needs.

2. Getting Started with Selenium

- **Topics to Cover:**
    - What is Selenium? (Architecture, WebDriver basics)
    - **Installation/setup**: Installing Selenium for Python (using pip, setting up drivers)
- **Depth:** Know the core components, be able to set up a Selenium project and run a basic script.

3. Selenium Locators & Interacting with Web Elements

- **Topics to Cover:**
    - Types of locators: ID, name, class, tag, CSS selectors, XPath
    - Interacting with elements: click, send_keys, clear, get text, dropdown selection, checkboxes, radio buttons
- **Depth:** Ability to select elements using multiple strategies and perform typical interactions in web apps. Be comfortable writing scripts to automate simple tasks.

4. Assertions and Validations

- **Topics to Cover:**
    - Using assert statements in Python
    - Common assertions in automated testing (e.g., title contains, element is visible, text exists)
- **Depth:** Write tests with meaningful validations, know how to verify UI and functional behavior.

5. Writing Your First Automation Scripts

- **Topics to Cover:**

    - Automate basic workflows (e.g., login, logout, form submission)
    - Handle waits (implicit/explicit wait) to make scripts robust
- **Depth:** Create scripts from scratch for standard web app features. Scripts should be readable, maintainable, and re-runnable.

6. Basic Page Object Model (POM)

- **Topics to Cover:**
    - What is POM? Why use it?
    - How to separate page locators and actions into classes
    - Refactor simple scripts into Page Objects for reusability
- **Depth:** Be able to convert procedural scripts into POM style. Understand how this improves test maintenance and scalability.

7. Hands-On Practice/Conversion of Manual Test Cases

- **Activities:**
    - Pick some manual test cases from your previous project (Project 1)
    - Convert them into automated Selenium Python scripts
    - Focus on real-world scenarios like logins, form filling, and validation checks
- **Depth:** Scripts should be reliable and demonstrate understanding of all concepts above—feel free to iterate and refactor.

Recommended Study Flow

1. **Begin with concepts and benefits** (1-2 hours)
2. **Move to Selenium setup and basic scripts** (4-5 hours)
3. **Deep dive into locators and interactions** (8-10 hours, with hands-on practice)
4. **Learn assertions and validations** (2-3 hours)
5. **Write and refine end-to-end scripts** (10-12 hours, including applying POM)

6. **Spend the rest practicing by converting manual tests** (remaining hours)

Aim: By the end of Week 2, you should be able to automate simple test cases for a web app with Selenium and Python, following best practices like using assertions and structuring code with the Page Object Model. This will build a strong foundation for more advanced concepts in later weeks.

Week – 3

For Week 3 of your 4-week course (aimed at transitioning from developer to QA), you'll focus on **automation frameworks and API testing**. Here's a clear breakdown of the topics you should study and the depth to which you need to go, tailored to your goal of building practical skills and your background as a developer:

1. Improving Selenium Automation

- **Study Areas:**
    - **Test Structure:** Organizing your test code into logical layers (tests, page objects, utilities).
    - **Data-Driven Testing:** Running your tests with different sets of input data (using frameworks like pytest parametrize or data from CSV/JSON).
- **Depth:**
    - Understand and implement the Page Object Model (POM).
    - Implement parameterized tests to run scenarios with multiple sets of data.
    - Refactor existing Selenium scripts from previous weeks for better structure and reusability.

2. Introduction to PyTest/UnitTest for Test Orchestration

- **Study Areas:**
    - **Basic concepts of PyTest/unittest:** Fixtures, test discovery, setup/teardown.
    - **Running & organizing tests:** Grouping tests, marking (tags like smoke, regression), skipping/xfail.
- **Depth:**

- Should be able to convert scripts into proper test functions/classes.
- Use fixtures for environment setup/teardown.
- Familiarity with running tests from command line and generating simple test reports.

3. API Testing with Postman

- **Study Areas:**
  - **Basic operations:** Making GET, POST, PUT, DELETE requests.
  - **Assertions:** Writing simple tests (e.g., status code, response body, headers).
  - **Chaining requests:** Passing data from one request to another using environment variables or scripting.
- **Depth:**
  - Be comfortable creating collections and writing tests in the "Tests" tab.
  - Must be able to document and share collections.
  - Be able to automate chaining 2-3 requests and validate their interdependency.

4. Building a Mini-Framework (Project-focused)

- **Study Areas:**
  - **Framework Structure:** How to structure folders/files for your automation suite.
  - **Reusability:** Utility methods for common API validations, config management.
  - **Documentation:** How to document test cases, endpoints covered, and test results.
- **Depth:**
  - Build a working mini-framework (either for Selenium or API, or ideally both).
  - Ensure it can be extended with new tests easily.
  - Provide a basic README or code comments documenting usage and coverage.

5. API Test Suite on Public APIs

- **Study Areas:**
    - **Hands-on Practice:** Test endpoints on APIs like reqres.in or openweathermap.org.
    - **Assertions & Coverage:** Cover different HTTP methods, edge cases (e.g., invalid input), and response validation.
- **Depth:**
    - Write tests for at least 5-7 different API endpoints.
    - Assert multiple response parameters for each test.
    - Measure and document which endpoints/scenarios are covered.

How Deep to Go (Summary)

- Aim for **practical, working scripts** over theoretical depth.
- Refactor and modularize for reusability and maintainability.
- Practice API and UI automation both, with priorities based on your interests and goals.
- Document your work at each stage, including test coverage and how to run your tests.
- By end of the week, you should have a **mini-framework**, a **set of automated tests** (UI + API), and a **simple report/documentation** showing what you've achieved.

This sets you up for a strong QA automation foundation as you near the end of your 4-week transition program.

Week – 4

For Week 4 of your Dev-to-QA transition course, the focus is on "DevOps" mindset: integrating test automation into CI/CD pipelines, effective reporting, QA best practices, and preparing your resume and portfolio. Here are the topics you need to study—with guidance on depth and focus, tailored for someone with your background and job search goals:

1. CI/CD Integration for Automation

- **CI/CD Tools:** Study both GitHub Actions and Azure Pipelines. Learn:
    - How to set up workflows/pipelines that trigger on code pushes/PRs.

    - How to add steps for installing dependencies, running test scripts, and collecting artifacts.
    - How to debug pipeline failures from logs.
- **Automation Script Integration:** Practice adding your automation (unit, integration, API tests) as steps in CI/CD.
- **Hands-on Goal:** By end of week, your automation suite should run automatically in CI/CD when code is updated.

2. Automated Test Reporting

- **Report Formats:** Learn how to generate and use HTML and JUnit/XML reports from your tests (Python: pytest-html, unittest-junitxml, etc.).
- **Publishing Reports:** Learn how to publish artifacts in both GitHub Actions and Azure Pipelines so you (and teammates/interviewers) can view/download results.
- **Best Practices:** Ensure your test reports highlight passes, failures, skipped/flaky tests, and execution timing.

3. QA Best Practices

- **Test Coverage:**
    - Learn about code coverage tools (coverage.py, pytest-cov, Coverage in .NET, etc.).
    - Understand what's meant by "good" coverage (~80%+), and limitations (not all coverage means quality tests).
- **Test Flakiness:**
    - Learn common causes (unstable environments, random inputs, dependent tests).
    - Learn mitigation: retries, fixing flaky logic, resource isolation.
- **Environment Setup:**
    - Practice setting up local/CI environments with consistent configs, secrets, and dependencies.
    - Learn to use Docker where possible to avoid "it works on my machine" issues.

4. Resume, Documentation, and Portfolio Polish

- **Resume Updates:** Tailor resume for QA/DevOps roles, highlighting:
    - Automation scripting experience, frameworks/tools you've mastered.
    - CI/CD pipeline integration.
    - Impact/results (coverage %, bugs found, speedups).
- **Project Documentation:** For your projects:
    - **Project 1:** "Manual & Automated Testing for [App Name]"
    - **Project 2:** "API Test Suite for [API Name]"
    - Ensure clear README: what, why, how to run tests, sample reports/screenshots, tech stack, brief coverage numbers.
- **Portfolio:**
    - Commit all code, reports, and docs to GitHub. Use clear project structure.
    - Add badges for build status, code coverage if possible.
- **Interview Prep:** Practice concise, confident answers about your end-to-end QA automation, CI/CD, reporting, and troubleshooting experience.

5. Demo and Practice

- **Demo:** Prepare to screen-share and walk through:
    - Pipelines running your tests automatically.
    - Reviewing test reports/artifacts.
    - Explaining how you made tests robust, maintainable, and report-friendly.
- **Mock Questions:** Practice explaining how you set up, run, debug, and report on your automation suite via CI/CD—this is key for interviews.

How Deep?

- Aim for practical, hands-on ability for **pipeline setup, automation script integration, and reporting.**
- For best practices, be ready to explain and show examples but don't dive into academic theory—focus on showing applied knowledge.
- For documentation and resume, produce results you'd be confident to share publicly or submit for QA job applications.

By the end of Week 4, you should have:

- End-to-end automation running in CI/CD (GitHub Actions/Azure Pipelines).
- Automatically generated test reports.
- Well-documented projects on GitHub, with a resume tailored to automation/QA.
- Comfort explaining your pipeline, reporting, and best practices for interviews.